시작하십시오
Astra Control Center

NetApp
July 19, 2022

This PDF was generated from https://docs.netapp.com/ko-kr/astra-control-center-2108/get-started/requirements.html on July 19, 2022. Always check docs.netapp.com for the latest.

목차

시작하십시오 1
Astra Control Center 요구 사항 1
Astra Control Center를 빠르게 시작합니다 3
Astra Control Center를 설치합니다 5
Astra Control Center를 설정합니다 16
Astra Control Center에 대한 질문과 대답 30

시작하십시오

Astra Control Center 요구 사항

Kubernetes 클러스터, 앱, 라이센스 및 웹 브라우저에 대한 지원을 확인하여 시작하십시오.

Kubernetes 클러스터 일반 요구사항

Kubernetes 클러스터는 Astra Control Center에서 검색 및 관리할 수 있도록 다음과 같은 일반 요구사항을 충족해야 합니다.

- * 이미지 레지스트리 *: Astra Control Center 빌드 이미지를 푸시할 수 있는 기존 개인 Docker 이미지 레지스트리가 있어야 합니다. 이미지를 업로드할 이미지 레지스트리의 URL이 있어야 하며 개인 컨테이너 레지스트리의 이미지에 태그를 지정해야 합니다.
- * Trident/ONTAP 스토리지 구성 *:Astra Control Center에서는 Trident 버전 21.01 또는 21.04가 이미 설치되어 스토리지 백엔드로 NetApp ONTAP 버전 9.5 이상에서 작동하도록 구성해야 합니다. Astra Control Center에서는 스토리지 클래스를 생성하고 기본 스토리지 클래스로 설정해야 합니다. Astra Control Center는 Trident에서 제공하는 다음과 같은 ONTAP 드라이버를 지원합니다.
  - ONTAP - NAS
  - ONTAP-NAS-Flexgroup입니다
  - ONTAP-SAN
  - ONTAP-SAN - 경제성

Astra Control Center에서 Kubernetes 클러스터를 관리하고 클러스터를 사용하여 Astra Control Center 설치를 호스트하려는 경우 클러스터에 다음과 같은 추가 요구사항이 있습니다.

- Kubernetes의 최신 버전입니다 "스냅샷 컨트롤러 구성 요소입니다" 이(가) 설치되었습니다
- 트라이던트 "볼륨스냅샷 클래스 개체" 관리자가 정의했습니다
- 클러스터에 기본 Kubernetes 스토리지 클래스가 있습니다
- Trident를 사용하도록 스토리지 클래스가 하나 이상 구성되었습니다
- Astra Control Center의 FQDN을 Astra Control Center 서비스의 외부 IP 주소로 가리키는 방법

OpenShift 클러스터

Astra Control Center에는 ONTAP 9.5 이상 버전에서 지원하는 Trident 스토리지 클래스가 있는 Red Hat OpenShift Container Platform 4.6.8 또는 4.7 클러스터가 필요합니다. 이러한 특성은 다음과 같습니다.

- 최소 300GB의 가용 ONTAP 스토리지 용량
- 3개의 컨트롤러 노드에서 4개의 CPU 코어, 16GB RAM 및 120GB의 사용 가능한 스토리지
- 최소 12개의 CPU 코어, 32GB RAM 및 50GB의 가용 스토리지를 갖춘 3개의 작업자 노드
- Kubernetes 버전 1.19 또는 1.20
- OpenShift 클러스터의 서비스로 수신 트래픽을 보내는 데 사용할 수 있는 서비스 유형 "로드 밸런서"입니다
- Astra Control Center의 FQDN을 부하 분산 IP 주소로 가리키는 방법

이러한 최소 요구 사항은 Astra Control Center가 OpenShift 클러스터에서 실행되는 유일한 애플리케이션이라고 가정합니다. 클러스터에서 추가 애플리케이션이 실행 중인 경우 이러한 최소 요구사항을 적절히 조정해야 합니다.

클러스터가 최소 요구사항을 충족하고 Kubernetes 모범 사례를 따르는 경우 Kubernetes 클러스터에서 Astra Control Center의 가용성이 높도록 하십시오.

OpenShift 4.8은 지원되지 않습니다.

애플리케이션 복제 중에 Astra Control Center는 OpenShift가 볼륨을 마운트하고 파일 소유권을 변경할 수 있도록 허용해야 합니다. 따라서 다음 명령을 사용하여 볼륨 작업을 성공적으로 완료할 수 있도록 ONTAP를 구성해야 합니다.

1. 'export-policy rule modify -vserver svm0 -policyname default-ruleindex 1 -superuser sys'
2. 수출정책규칙 수정-정책명 default-ruleindex 1-anon 65534

두 번째 OpenShift 4.6 또는 4.7 클러스터를 관리되는 컴퓨팅 리소스로 추가하려면 Trident Volume Snapshot 기능이 활성화되어 있는지 확인해야 합니다. 공식 Trident를 참조하십시오 "지침" Trident를 사용하여 볼륨 스냅샷을 활성화하고 테스트합니다.

설명합니다

Astra Control Center에는 다음과 같은 앱 관리 요구 사항이 있습니다.

- * 라이선스 *: Astra Control Center를 사용하여 앱을 관리하려면 Astra Control Center 라이센스가 필요합니다.
- * Helm 3 *: Helm을 사용하여 앱을 배포하는 경우 Astra Control Center에 Helm 버전 3이 필요합니다. Helm 3으로 배포된 애플리케이션 관리 및 복제(또는 Helm 2에서 Helm 3으로 업그레이드)가 완벽하게 지원됩니다. Helm 2와 함께 배포된 앱은 지원되지 않습니다.
- * 운영자 관리 *: Astra Control Center는 OLM(Operator Lifecycle Manager) 지원 운영자 또는 클러스터 범위 연산자와 함께 배포된 앱을 지원하지 않습니다.

인터넷 접속

인터넷에 대한 외부 액세스 권한이 있는지 확인해야 합니다. 그렇지 않으면 NetApp Cloud Insights에서 모니터링 및 메트릭 데이터를 수신하거나 지원 번들을 NetApp Support 사이트로 보내는 등 일부 기능이 제한될 수 있습니다.

라이센스

Astra Control Center의 모든 기능을 사용하려면 Astra Control Center 라이센스가 필요합니다. NetApp에서 평가판 라이센스 또는 전체 라이센스를 받으십시오. 라이센스가 없으면 다음을 수행할 수 없습니다.

- 사용자 지정 앱 정의
- 기존 앱의 스냅샷 또는 클론 생성
- 데이터 보호 정책을 구성합니다

Astra Control Center를 사용해 보고 싶다면 가능합니다 "90일 평가판 라이센스를 사용합니다".

온프레미스 Kubernetes 클러스터의 서비스 유형 "로드 밸런서

Astra Control Center는 "loadbalancer"(Astra Control Center 네임스페이스의 svc/traefik) 유형의 서비스를 사용하며 액세스 가능한 외부 IP 주소를 할당해야 합니다. 온-프레미스 OpenShift 클러스터의 경우 를 사용할 수 있습니다 "메탈리스" 외부 IP 주소를 서비스에 자동으로 할당합니다. 내부 DNS 서버 구성에서 Astra Control Center에 대해 선택한 DNS 이름을 부하 분산 IP 주소로 지정해야 합니다.

네트워킹 요구 사항

Astra Control Center를 호스팅하는 클러스터는 다음 TCP 포트를 사용하여 통신합니다. 이러한 포트가 모든 방화벽을 통해 허용되는지 확인하고 Astra 네트워크에서 발생하는 HTTPS 송신 트래픽을 허용하도록 방화벽을 구성해야 합니다. 일부 포트에는 Astra Control Center를 호스팅하는 클러스터와 각 관리 클러스터(해당되는 경우) 간의 연결이 모두 필요합니다.

제품	포트	프로토콜	방향	목적
Astra 제어 센터	443	HTTPS	침투	UI/API 액세스 - Astra Control Center를 호스팅하는 클러스터와 관리되는 각 클러스터 간에 이 포트가 열려 있는지 확인합니다
Astra 제어 센터	9090	HTTPS	• 수신(Astra Control Center를 호스팅하는 클러스터에) • 송신(각 관리 클러스터의 각 작업자 노드의 노드 IP 주소에서 임의 포트)	메트릭 데이터를 고객에게 제공 - 각 관리 클러스터가 Astra Control Center를 호스팅하는 클러스터에서 이 포트에 액세스할 수 있는지 확인합니다
트라이던트	34571	HTTPS	침투	노드 POD 통신
트라이던트	9220	HTTP	침투	메트릭 엔드포인트

지원되는 웹 브라우저

Astra Control Center는 1280 x 720의 최소 해상도로 최신 버전의 Firefox, Safari 및 Chrome을 지원합니다.

다음 단계

를 봅니다 "빠른 시작" 개요.

Astra Control Center를 빠르게 시작합니다

이 페이지에서는 Astra Control Center를 시작하는 데 필요한 단계에 대해 개괄적으로 설명합니다. 각 단계의 링크를 클릭하면 자세한 내용을 제공하는 페이지로 이동합니다.

사용해 보세요! Astra Control Center를 사용해 보고 싶은 경우 90일 평가판 라이센스를 사용할 수 있습니다. 을 참조하십시오 "라이센스 정보" 를 참조하십시오.

span class="image"><img src="<a href="https://raw.githubusercontent.com/NetAppDocs/common/main/media/number-1.png"" class="bare">https://raw.githubusercontent.com/NetAppDocs/common/main/media/number-1.png"</a> Alt="One"></span> Kubernetes 클러스터 요구 사항을 검토합니다

- Astra는 Trident에서 구성한 ONTAP 스토리지 백엔드를 통해 Kubernetes 클러스터와 함께 작동합니다.
- 클러스터는 정상 상태에서 실행되어야 하며 3개 이상의 온라인 작업자 노드가 있어야 합니다.
- 클러스터가 Kubernetes를 실행 중이어야 합니다.

"Astra Control Center 요구 사항에 대해 자세히 알아보십시오".

span class="image"><img src="<a href="https://raw.githubusercontent.com/NetAppDocs/common/main/media/number-2.png"" class="bare">https://raw.githubusercontent.com/NetAppDocs/common/main/media/number-2.png"</a> Alt="Two"></span>Astra Control Center를 다운로드하여 설치합니다

- NetApp Support 사이트에서 Astra Control Center를 다운로드하십시오.
- 현지 환경에 Astra Control Center를 설치합니다.
- ONTAP 스토리지 백엔드에서 지원하는 Trident 구성을 확인해 보십시오.

  첫 번째 릴리스에서는 OpenShift 레지스트리에 이미지를 설치하거나 로컬 레지스트리를 사용합니다.

"Astra Control Center 설치에 대해 자세히 알아보십시오".

span class="image"><img src="<a href="https://raw.githubusercontent.com/NetAppDocs/common/main/media/number-3.png"" class="bare">https://raw.githubusercontent.com/NetAppDocs/common/main/media/number-3.png"</a> Alt="3"></span>일부 초기 설정 작업을 완료합니다

- 라이센스를 추가합니다.
- Kubernetes 클러스터 추가 및 Astra Control Center에서 세부 정보를 검색합니다.
- ONTAP 스토리지 백엔드를 추가합니다.
- 필요에 따라 앱 백업을 저장할 오브젝트 저장소 버킷을 추가합니다.

"초기 설정 프로세스에 대해 자세히 알아보십시오".

Astra Control Center 설정을 마치면 다음 단계를 수행하십시오.

- 앱을 관리합니다. "앱 관리 방법에 대해 자세히 알아보십시오".
- 필요한 경우 NetApp Cloud Insights에 연결하여 Astra Control Center UI 내에서 시스템 상태, 용량 및 처리량에 대한 메트릭을 표시할 수 있습니다. "Cloud Insights에 연결하는 방법에 대해 자세히 알아보십시오".

"Astra Control Center를 설치합니다".

자세한 내용을 확인하십시오

- "Astra API를 사용합니다"

Astra Control Center를 설치합니다

Astra Control Center를 설치하려면 다음 단계를 수행하십시오.

- [Install Astra Control Center]
- [Log in to the Astra Control Center UI]

Astra Control Center를 설치합니다

Astra Control Center를 설치하려면 NetApp 지원 사이트에서 설치 번들을 다운로드하고 일련의 명령을 실행하여 사용자 환경에 Astra Control Center Operator 및 Astra Control Center를 설치합니다. 이 절차를 사용하여 인터넷에 연결되었거나 공기가 연결된 환경에 Astra Control Center를 설치할 수 있습니다.

무엇을 '필요로 할거야

- "설치를 시작하기 전에 Astra Control Center 구축을 위한 환경을 준비합니다".
- OpenShift 클러스터에서 모든 클러스터 운영자가 정상 상태인지 확인합니다('사용 가능'은 '참'임).

```
oc get clusteroperators
```

- OpenShift 클러스터에서 모든 API 서비스가 정상 상태인지 확인합니다('사용 가능'은 '참'임).

```
oc get apiservices
```

Astra Control Center 설치 프로세스는 다음을 수행합니다.

- "NetApp-acc"(또는 사용자 지정 이름) 네임스페이스에 Astra 구성 요소를 설치합니다.
- 기본 계정을 만듭니다.
- Astra Control Center의 이 인스턴스에 대해 기본 관리 사용자 이메일 주소와 기본 1회 암호 ACC-<UUID_of_installation>(ACC-<UUID_of_installation>)를 설정합니다. 이 사용자에게는 시스템에서 소유자 역할이 할당되며 UI에 처음 로그인할 때 필요합니다.
- 모든 Astra Control Center Pod가 실행 중인지 확인하는 데 도움이 됩니다.
- Astra UI를 설치합니다.

> Red Hat의 Podman 리포지토리를 사용하는 경우 docker 명령 대신 Podman 명령을 사용할 수 있습니다.

단계

1. 에서 Astra Control Center 번들('Astra-control-center-[version].tar.gz')을 다운로드합니다 "NetApp Support 사이트".
2. Astra Control Center 인증서 및 키의 지퍼를 에서 다운로드하십시오 "NetApp Support 사이트".
3. (선택 사항) 다음 명령을 사용하여 번들의 서명을 확인합니다.

```
openssl dgst -sha256 -verify astra-control-center[version].pub
-signature <astra-control-center[version].sig astra-control-
center[version].tar.gz
```

4. 이미지 추출:

```
tar -vxzf astra-control-center-[version].tar.gz
```

5. Astra 디렉토리로 변경합니다.

```
cd astra-control-center-[version]
```

6. Astra Control Center 이미지 디렉토리의 파일을 로컬 레지스트리에 추가합니다.

   아래 이미지의 자동 로드에 대한 샘플 스크립트를 참조하십시오.

   a. Docker 레지스트리에 로그인합니다.

   ```
   docker login [Docker_registry_path]
   ```

   b. Docker에 이미지를 로드합니다.
   c. 이미지에 태그를 지정합니다.
   d. 이미지를 로컬 레지스트리에 밀어 넣습니다.

```
export REGISTRY=[Docker_registry_path]
for astraImageFile in $(ls images/*.tar) ; do
  # Load to local cache. And store the name of the loaded image trimming
the 'Loaded images: '
  astraImage=$(docker load --input ${astraImageFile} | sed 's/Loaded
image: //')
  astraImage=$(echo ${astraImage} | sed 's!localhost/!!')
  # Tag with local image repo.
  docker tag ${astraImage} ${REGISTRY}/${astraImage}
  # Push to the local repo.
  docker push ${REGISTRY}/${astraImage}
done
```

7. (인증 요구 사항이 있는 레지스트리 전용) 인증이 필요한 레지스트리를 사용하는 경우 다음을 수행해야 합니다.

   a. 'NetApp-acc-operator' 네임스페이스 생성:

```
kubectl create ns netapp-acc-operator
```

응답:

```
namespace/netapp-acc-operator created
```

b. NetApp-acc-operator 네임스페이스에 대한 암호를 생성합니다. Docker 정보를 추가하고 다음 명령을 실행합니다.

```
kubectl create secret docker-registry astra-registry-cred -n netapp-
acc-operator --docker-server=[Docker_registry_path] --docker
-username=[username] --docker-password=[token]
```

샘플 반응:

```
secret/astra-registry-cred created
```

c. "NetApp-acc"(또는 사용자 지정 이름) 네임스페이스를 생성합니다.

```
kubectl create ns [netapp-acc or custom]
```

샘플 반응:

```
namespace/netapp-acc created
```

d. "NetApp-acc"(또는 사용자 지정 이름) 네임스페이스에 대한 암호를 생성합니다. Docker 정보를 추가하고 다음 명령을 실행합니다.

```
kubectl create secret docker-registry astra-registry-cred -n [netapp-
acc or custom] --docker-server=[Docker_registry_path] --docker
-username=[username] --docker-password=[token]
```

응답

```
secret/astra-registry-cred created
```

8. Astra Control Center 운영자 배포 YAML('Astra_control_center_operator_deploy.YAML')을 편집하여 현지 등록부와 비밀을 참조하십시오.

```
vim astra_control_center_operator_deploy.yaml
```

a. 인증이 필요한 레지스트리를 사용하는 경우 'imagePullSecrets:[]'의 기본 줄을 다음과 같이 바꿉니다.

```
imagePullSecrets:
- name: astra-registry-cred
```

b. kuby-RBAC-Prox 이미지의 [docker_registry_path]를 이전 단계에서 이미지를 푸시한 레지스트리 경로로 변경합니다.

c. "acc-operator-controller-manager" 이미지의 [docker_registry_path]를 이전 단계에서 이미지를 푸시한 레지스트리 경로로 변경합니다.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    control-plane: controller-manager
  name: acc-operator-controller-manager
  namespace: netapp-acc-operator
spec:
  replicas: 1
  selector:
    matchLabels:
      control-plane: controller-manager
  template:
    metadata:
      labels:
        control-plane: controller-manager
    spec:
      containers:
      - args:
        - --secure-listen-address=0.0.0.0:8443
        - --upstream=http://127.0.0.1:8080/
        - --logtostderr=true
        - --v=10
        image: [Docker_registry_path]/kube-rbac-proxy:v0.5.0
        name: kube-rbac-proxy
        ports:
        - containerPort: 8443
          name: https
      - args:
        - --health-probe-bind-address=:8081
        - --metrics-bind-address=127.0.0.1:8080
        - --leader-elect
        command:
        - /manager
        env:
        - name: ACCOP_LOG_LEVEL
          value: "2"
        image: [Docker_registry_path]/acc-operator:[version x.y.z]
        imagePullPolicy: IfNotPresent
      imagePullSecrets: []
```

9. Astra Control Center 사용자 정의 리소스(CR) 파일('Astra_control_center_min YAML') 편집:

```
vim astra_control_center_min.yaml
```

> 사용자 환경에 추가 사용자 정의가 필요한 경우 대체 CR로 Astra_control_center.yaML을 사용할 수 있습니다. Astra_control_center_min YAML은 기본 CR이며 대부분의 설치에 적합합니다.

> CR에서 구성한 속성은 초기 Astra Control Center 배포 후에는 변경할 수 없습니다.

a. 이전 단계에서 이미지를 푸시한 레지스트리 경로로 '[docker_registry_path]'를 변경합니다.

b. accountName 문자열을 계정과 연결할 이름으로 변경합니다.

c. Astra에 액세스하기 위해 브라우저에서 사용할 FQDN으로 "astraAddress" 문자열을 변경합니다. 주소에 http:// 또는 https:// 를 사용하지 마십시오. 에서 사용하기 위해 이 FQDN을 복사합니다 나중에.

d. e-메일 문자열을 기본 초기 관리자 주소로 변경합니다. 에서 사용할 이 이메일 주소를 복사합니다 나중에.

e. 인터넷 연결이 없는 사이트의 경우 AutoSupport에 등록된 사이트를 거짓으로 변경하거나 연결된 사이트의 경우 "참"으로 변경합니다.

f. (선택 사항) 계정과 연결된 사용자의 이름 "FirstName"과 성 "LastName"을 추가합니다. UI 내에서 이 단계를 지금 또는 나중에 수행할 수 있습니다.

g. (선택 사항) 설치에 필요한 경우 'torageClass' 값을 다른 Trident storageClass 리소스로 변경합니다.

h. 인증이 필요한 레지스트리를 사용하지 않는 경우 '비밀' 줄을 삭제합니다.

```
apiVersion: astra.netapp.io/v1
kind: AstraControlCenter
metadata:
  name: astra
spec:
  accountName: "Example"
  astraVersion: "ASTRA_VERSION"
  astraAddress: "astra.example.com"
  autoSupport:
    enrolled: true
  email: "[admin@example.com]"
  firstName: "SRE"
  lastName: "Admin"
  imageRegistry:
    name: "[Docker_registry_path]"
    secret: "astra-registry-cred"
  storageClass: "ontap-gold"
```

10. Astra Control Center 운영자를 설치합니다.

```
kubectl apply -f astra_control_center_operator_deploy.yaml
```

샘플 반응:

```
namespace/netapp-acc-operator created
customresourcedefinition.apiextensions.k8s.io/astracontrolcenters.astra.
netapp.io created
role.rbac.authorization.k8s.io/acc-operator-leader-election-role created
clusterrole.rbac.authorization.k8s.io/acc-operator-manager-role created
clusterrole.rbac.authorization.k8s.io/acc-operator-metrics-reader
created
clusterrole.rbac.authorization.k8s.io/acc-operator-proxy-role created
rolebinding.rbac.authorization.k8s.io/acc-operator-leader-election-
rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/acc-operator-manager-
rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/acc-operator-proxy-
rolebinding created
configmap/acc-operator-manager-config created
service/acc-operator-controller-manager-metrics-service created
deployment.apps/acc-operator-controller-manager created
```

11. 이전 단계에서 작성하지 않은 경우, "NetApp-acc"(또는 사용자 지정) 네임스페이스를 작성하십시오.

```
kubectl create ns [netapp-acc or custom]
```

샘플 반응:

```
namespace/netapp-acc created
```

12. 다음 패치를 실행하여 수정합니다 "클러스터 역할 바인딩".
13. "NetApp-acc"(또는 사용자 지정) 네임스페이스에 Astra Control Center를 설치합니다.

```
kubectl apply -f astra_control_center_min.yaml -n [netapp-acc or custom]
```

샘플 반응:

```
astracontrolcenter.astra.netapp.io/astra created
```

14. 모든 시스템 구성 요소가 성공적으로 설치되었는지 확인합니다.

```
kubectl get pods -n [netapp-acc or custom]
```

각 포드는 'Running' 상태여야 합니다. 시스템 포드를 구축하는 데 몇 분 정도 걸릴 수 있습니다.

샘플 반응:

```
NAME                                          READY   STATUS    RESTARTS
AGE
acc-helm-repo-5fdfff786f-gkv6z                1/1     Running   0
4m58s
activity-649f869bf7-jn5gs                     1/1     Running   0
3m14s
asup-79846b5fdc-s9s97                         1/1     Running   0
3m10s
authentication-84c78f5cf4-qhx9t               1/1     Running   0
118s
billing-9b8496787-v8rzv                       1/1     Running   0
2m54s
bucketservice-5fb876d9d5-wkfvz                1/1     Running   0
3m26s
cloud-extension-f9f4f59c6-dz6s6               1/1     Running   0
3m
cloud-insights-service-5676b8c6d4-6q7lv       1/1     Running   0
2m52s
composite-compute-7dcc9c6d6c-lxdr6            1/1     Running   0
2m50s
composite-volume-74dbfd7577-cd42b             1/1     Running   0
3m2s
credentials-75dbf46f9d-5qm2b                  1/1     Running   0
3m32s
entitlement-6cf875cb48-gkvhp                  1/1     Running   0
3m12s
features-74fd97bb46-vss2n                     1/1     Running   0
3m6s
fluent-bit-ds-2g9jb                           1/1     Running   0
113s
fluent-bit-ds-5tg5h                           1/1     Running   0
113s
fluent-bit-ds-qfxb8                           1/1     Running   0
113s
graphql-server-7769f98b86-p4qrv               1/1     Running   0
90s
identity-566c566cd5-ntfj6                     1/1     Running   0
3m16s
influxdb2-0                                   1/1     Running   0
4m43s
krakend-5cb8d56978-44q66                      1/1     Running   0
93s
license-66cbbc6f48-27kgf                      1/1     Running   0
3m4s
```

```
login-ui-584f7fd84b-dmdrp                    1/1     Running   0
87s
loki-0                                       1/1     Running   0
4m44s
metrics-ingestion-service-6dcfddf45f-mhnvh   1/1     Running   0
3m8s
monitoring-operator-78d67b4d4-nxs6v          2/2     Running   0
116s
nats-0                                       1/1     Running   0
4m40s
nats-1                                       1/1     Running   0
4m26s
nats-2                                       1/1     Running   0
4m15s
nautilus-9b664bc55-rn9t8                     1/1     Running   0
2m56s
openapi-dc5ddfb7d-6q8vh                      1/1     Running   0
3m20s
polaris-consul-consul-5tjs7                  1/1     Running   0
4m43s
polaris-consul-consul-5wbnx                  1/1     Running   0
4m43s
polaris-consul-consul-bfvl7                  1/1     Running   0
4m43s
polaris-consul-consul-server-0               1/1     Running   0
4m43s
polaris-consul-consul-server-1               1/1     Running   0
4m43s
polaris-consul-consul-server-2               1/1     Running   0
4m43s
polaris-mongodb-0                            2/2     Running   0
4m49s
polaris-mongodb-1                            2/2     Running   0
4m22s
polaris-mongodb-arbiter-0                    1/1     Running   0
4m49s
polaris-ui-6648875998-75d98                  1/1     Running   0
92s
polaris-vault-0                              1/1     Running   0
4m41s
polaris-vault-1                              1/1     Running   0
4m41s
polaris-vault-2                              1/1     Running   0
4m41s
storage-backend-metrics-69546f4fc8-m7lfj     1/1     Running   0
3m22s
```

```
storage-provider-5d46f755b-qfv89             1/1     Running   0
3m30s
support-5dc579865c-z4pwq                     1/1     Running   0
3m18s
telegraf-ds-4452f                            1/1     Running   0
113s
telegraf-ds-gnqxl                            1/1     Running   0
113s
telegraf-ds-jhw74                            1/1     Running   0
113s
telegraf-rs-gg6m4                            1/1     Running   0
113s
telemetry-service-6dcc875f98-zft26           1/1     Running   0
3m24s
tenancy-7f7f77f699-q7l6w                     1/1     Running   0
3m28s
traefik-769d846f9b-c9crt                     1/1     Running   0
83s
traefik-769d846f9b-l9n4k                     1/1     Running   0
67s
trident-svc-8649c8bfc5-pdj79                 1/1     Running   0
2m57s
vault-controller-745879f98b-49c5v            1/1     Running   0
4m51s
```

15. (선택 사항) 설치가 완료되었는지 확인하려면 다음 명령을 사용하여 "acc-operator" 로그를 볼 수 있습니다.

```
kubectl logs deploy/acc-operator-controller-manager -n netapp-acc-
operator -c manager -f
```

16. 모든 Pod가 실행 중인 경우 ACC Operator가 설치한 AstraControlCenter 인스턴스를 검색하여 설치 성공 여부를 확인합니다.

```
kubectl get acc -o yaml -n netapp-acc
```

17. '구축' 값에 대한 응답으로 'tatus.deploymentState` 필드를 확인합니다. 배포에 실패한 경우 대신 오류 메시지가 나타납니다.

> 다음 단계에서 uuid를 사용합니다.

```
apiVersion: v1
items:
- apiVersion: astra.netapp.io/v1
  kind: AstraControlCenter
  metadata:
    creationTimestamp: "2021-07-28T21:36:49Z"
    finalizers:
    - astracontrolcenter.netapp.io/finalizer
   generation: 1
    name: astra
    namespace: netapp-acc
    resourceVersion: "27797604"
    selfLink: /apis/astra.netapp.io/v1/namespaces/netapp-
acc/astracontrolcenters/astra
    uid: 61cd8b65-047b-431a-ba35-510afcb845f1
  spec:
    accountName: Example
    astraAddress: astra.example.com
    astraResourcesScaler: "Off"
    astraVersion: 21.08.52
    autoSupport:
      enrolled: false
    email: admin@example.com
    firstName: SRE
    lastName: Admin
    imageRegistry:
      name: registry_name/astra
  status:
    certManager: deploy
    deploymentState: Deployed
    observedGeneration: 1
    observedVersion: 21.08.52
    postInstall: Complete
    uuid: c49008a5-4ef1-4c5d-a53e-830daf994116
kind: List
metadata:
  resourceVersion: ""
  selfLink: ""
```

18. Astra Control Center에 로그인할 때 사용할 1회 암호를 얻으려면 이전 단계의 응답에서 'Status.uuid' 값을 복사합니다. 암호는 ACC-, UUID 값(ACC-[UUID]), 이 예에서는 ACC-c49008a5-4ef1-4c5d-a53e-830daf994116) 순으로 입력된다.

Astra Control Center UI에 로그인합니다

ACC를 설치한 후 기본 관리자의 암호를 변경하고 ACC UI 대시보드에 로그인합니다.

단계

1. 브라우저에서, Astra_control_center_min YAML'cr when의 astraAddress에 사용한 FQDN을 입력한다 ACC를 설치했습니다.
2. 메시지가 표시되면 자체 서명된 인증서를 수락합니다.

   로그인 후 사용자 지정 인증서를 만들 수 있습니다.

3. Astra Control Center 로그인 페이지에서 Astra_control_center_min YAML CR when에 e-mail에 사용한 값을 입력합니다 ACC를 설치했습니다1회 암호('ACC-[UUID]')를 입력합니다.

   잘못된 암호를 세 번 입력하면 15분 동안 관리자 계정이 잠깁니다.

4. Login * 을 선택합니다.
5. 메시지가 나타나면 암호를 변경합니다.

   처음 로그인하는 데 암호를 잊은 경우 다른 관리 사용자 계정이 아직 생성되지 않은 경우 NetApp 지원에 암호 복구 지원을 문의하십시오.

6. (선택 사항) 기존의 자체 서명된 TLS 인증서를 제거하고 로 바꿉니다 "인증 기관(CA)에서 서명한 사용자 지정 TLS 인증서".

설치 문제를 해결합니다

서비스 중 '오류' 상태인 서비스가 있으면 로그를 검사할 수 있습니다. 400 ~ 500 범위의 API 응답 코드를 찾습니다. 이는 고장이 발생한 장소를 나타냅니다.

단계

1. Astra Control Center 운영자 로그를 검사하려면 다음을 입력하십시오.

```
kubectl logs --follow -n netapp-acc-operator $(kubectl get pods -n
netapp-acc-operator -o name)  -c manager
```

다음 단계

를 수행하여 배포를 완료합니다 "설정 작업".

Astra Control Center를 설정합니다

Astra Control Center를 설치하고, UI에 로그인하고, 암호를 변경하면 라이센스를 설정하고, 클러스터를 추가하고, 스토리지를 관리하고, 버킷을 추가할 수 있습니다.

작업

- [Add a license for Astra Control Center]
- [Add cluster]
- [Add a storage backend]
- [Add a bucket]

Astra Control Center에 대한 라이센스를 추가합니다

UI 또는 를 사용하여 새 라이센스를 추가할 수 있습니다 "API를 참조하십시오" Astra Control Center의 모든 기능을 활용할 수 있습니다. 라이센스가 없으면 Astra Control Center의 사용은 사용자 관리 및 새 클러스터 추가로 제한됩니다.

에서 Astra Control Center를 다운로드한 경우 "NetApp Support 사이트"또한 NetApp 라이센스 파일(NLF)도 다운로드했습니다. 이 라이센스 파일에 대한 액세스 권한이 있는지 확인하십시오.

> 기존 평가판 또는 전체 라이센스를 업데이트하려면 을 참조하십시오 "기존 라이센스를 업데이트합니다".

전체 또는 평가 라이센스를 추가합니다

Astra Control Center 라이센스는 Kubernetes CPU 장치를 사용하여 CPU 리소스를 측정합니다. 라이센스는 모든 관리되는 Kubernetes 클러스터의 작업자 노드에 할당된 CPU 리소스를 고려해야 합니다. 라이센스를 추가하기 전에 에서 라이센스 파일(NLF)을 얻어야 합니다 "NetApp Support 사이트".

또한 평가판 라이센스가 있는 Astra Control Center를 사용하여 라이센스를 다운로드한 날짜로부터 90일 동안 Astra Control Center를 사용할 수 있습니다. 등록하면 무료 평가판을 사용할 수 있습니다 "여기".

> 설치가 라이센스 CPU 유닛 수를 초과하여 증가할 경우, Astra Control Center를 통해 새 애플리케이션을 관리할 수 없습니다. 용량이 초과되면 경고가 표시됩니다.

단계

1. Astra Control Center UI에 로그인합니다.
2. 계정 * > * 라이센스 * 를 선택합니다.
3. 라이센스 추가 * 를 선택합니다.
4. 다운로드한 라이센스 파일(NLF)으로 이동합니다.
5. 라이센스 추가 * 를 선택합니다.

계정 * > * 라이센스 * 페이지에는 라이센스 정보, 만료 날짜, 라이센스 일련 번호, 계정 ID 및 사용된 CPU 단위가 표시됩니다.

> 평가 라이센스가 있는 경우 ASUP를 보내지 않을 경우 Astra Control Center에 장애가 발생할 경우 데이터 손실을 방지하기 위해 계정 ID를 저장해야 합니다.

클러스터 추가

앱 관리를 시작하려면 Kubernetes 클러스터를 추가하고 이를 컴퓨팅 리소스로 관리합니다. Kubernetes 애플리케이션을 검색하려면 Astra Control Center용 클러스터를 추가해야 합니다.

관리를 위해 Astra Control Center에 다른 클러스터를 추가하기 전에 먼저 Astra Control Center에서 클러스터를 관리하는 것이 좋습니다. 메트릭 및 문제 해결을 위해 Kubemetrics 데이터 및 클러스터 관련 데이터를 전송하려면 관리 중인 초기 클러스터가 필요합니다. 클러스터 추가 * 기능을 사용하여 Astra Control Center로 클러스터를 관리할 수 있습니다.

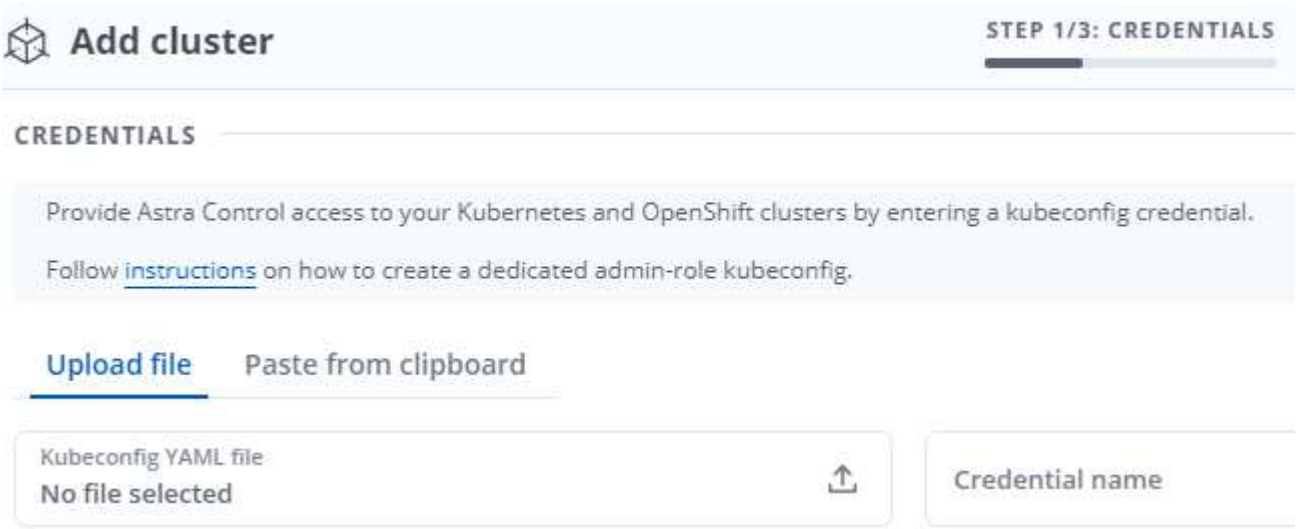
무엇을 '필요로 할거야

클러스터를 추가하기 전에 필요한 를 검토 및 수행합니다 "선행 작업".

단계

1. Astra Control Center UI의 * Dashboard * 에서 Clusters 섹션에서 * Add * 를 선택합니다.
2. Add Cluster * (클러스터 추가 *) 창이 열리면 kubecononfig.YAML 파일을 업로드하거나 kubecononfig.YAML 파일의 내용을 붙여 넣습니다.

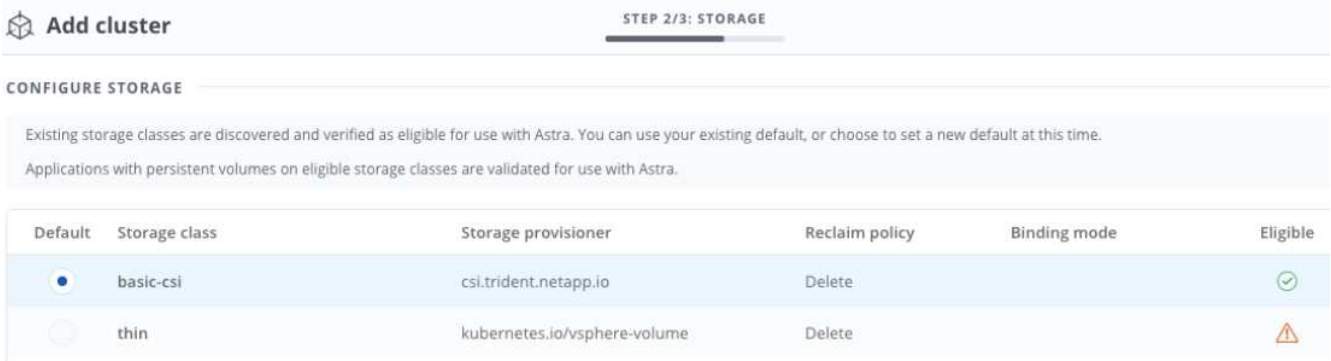
   "kubecononfig.yAML" 파일에는 하나의 클러스터에 대한 클러스터 자격 증명만 * 포함되어야 합니다.



   고유한 "kubecononfig" 파일을 만들 경우 해당 파일에 * 하나의 * 컨텍스트 요소만 정의해야 합니다. 을 참조하십시오 "Kubernetes 문서" kubecononfig 파일을 만드는 방법에 대한 자세한 내용은

3. 자격 증명 이름을 제공하십시오. 기본적으로 자격 증명 이름은 클러스터 이름으로 자동 채워집니다.
4. 스토리지 구성 * 을 선택합니다.
5. 이 Kubernetes 클러스터에 사용할 스토리지 클래스를 선택하고 * Review * 를 선택하십시오.

   ONTAP 스토리지가 지원하는 Trident 스토리지 클래스를 선택해야 합니다.



6. 정보를 검토하고 모든 내용이 양호하면 * 클러스터 추가 * 를 선택합니다.

클러스터가 * 검색 * 상태로 전환되고 * 실행 * 으로 변경됩니다. Kubernetes 클러스터를 성공적으로 추가했으며 현재 Astra Control Center에서 관리하고 있습니다.

> Astra Control Center에서 관리할 클러스터를 추가한 후 모니터링 연산자를 구축하는 데 몇 분이 걸릴 수 있습니다. 그 전까지는 알림 아이콘이 빨간색으로 바뀌고 * 모니터링 에이전트 상태 확인 실패 * 이벤트를 기록합니다. Astra Control Center가 올바른 상태를 획득하면 문제가 해결되므로 이 문제를 무시할 수 있습니다. 몇 분 이내에 문제가 해결되지 않으면 클러스터로 이동하여 "OC get Pod-n NetApp-monitoring"을 시작 지점으로 실행합니다. 문제를 디버깅하려면 모니터링 운영자 로그를 확인해야 합니다.

스토리지 백엔드를 추가합니다

Astra Control에서 리소스를 관리할 수 있도록 스토리지 백엔드를 추가할 수 있습니다. Astra Control에서 스토리지 클러스터를 스토리지 백엔드로 관리하면 PVS(영구적 볼륨)와 스토리지 백엔드 간의 연결 및 추가 스토리지 메트릭을 얻을 수 있습니다.

다음과 같은 방법으로 스토리지 백엔드를 추가할 수 있습니다.

- 클러스터를 추가할 때 스토리지를 구성합니다. 을 참조하십시오 "클러스터 추가".
- Dashboard 또는 Backend 옵션을 사용하여 검색된 스토리지 백엔드를 추가합니다.

다음 옵션을 사용하여 이미 검색된 스토리지 백엔드를 추가할 수 있습니다.

- [Add storage backend using Dashboard]
- [Add storage backend using Backends option]

대시보드를 사용하여 스토리지 백엔드를 추가합니다

1. 대시보드에서 다음 중 하나를 수행합니다.
   a. 대시보드 스토리지 백엔드 섹션에서 * 관리 * 를 선택합니다.
   b. 대시보드 리소스 요약 > 스토리지 백엔드 섹션에서 * 추가 * 를 선택합니다.
2. ONTAP 관리자 자격 증명을 입력하고 * 검토 * 를 선택합니다.
3. 백엔드 세부 정보를 확인하고 * 관리 * 를 선택합니다.

   백엔드가 요약 정보와 함께 목록에 나타납니다.

백엔드 옵션을 사용하여 스토리지 백엔드를 추가합니다

1. 왼쪽 탐색 영역에서 * backends * 를 선택합니다.
2. 관리 * 를 선택합니다.
3. ONTAP 관리자 자격 증명을 입력하고 * 검토 * 를 선택합니다.
4. 백엔드 세부 정보를 확인하고 * 관리 * 를 선택합니다.

   백엔드가 요약 정보와 함께 목록에 나타납니다.

5. 백엔드 스토리지에 대한 세부 정보를 보려면 해당 스토리지를 선택합니다.

   관리되는 컴퓨팅 클러스터의 애플리케이션에서 사용하는 영구 볼륨도 표시됩니다.

버킷을 추가합니다

애플리케이션과 영구 스토리지를 백업하려는 경우나 클러스터 간에 애플리케이션을 클론 복제하려는 경우에는 오브젝트 저장소 버킷 공급자를 추가하는 것이 중요합니다. Astra Control은 이러한 백업 또는 클론을 정의한 오브젝트 저장소 버킷에 저장합니다.

버킷을 추가하면 Astra Control은 하나의 버킷을 기본 버킷 표시기로 표시합니다. 사용자가 만든 첫 번째 버킷이 기본 버킷이 됩니다.

애플리케이션 구성과 영구 스토리지를 동일한 클러스터에 클론 복제할 경우 버킷이 필요하지 않습니다.

다음 버킷 유형 중 하나를 사용하십시오.

- NetApp ONTAP S3
- NetApp StorageGRID S3
- 일반 S3

Astra Control Center는 Amazon S3를 일반 S3 버킷 공급자로 지원하지만, Astra Control Center는 Amazon의 S3 지원을 주장하는 모든 오브젝트 저장소 공급업체를 지원하지 않을 수 있습니다.

Astra API를 사용하여 버킷을 추가하는 방법에 대한 지침은 를 참조하십시오 "Astra 자동화 및 API 정보".

단계

1. 왼쪽 탐색 영역에서 * Bucket * 을 선택합니다.
   a. 추가 * 를 선택합니다.
   b. 버킷 유형을 선택합니다.

      버킷을 추가할 때 해당 공급자에 맞는 자격 증명으로 올바른 버킷 공급자 유형을 선택합니다. 예를 들어, UI에서 NetApp ONTAP S3를 StorageGRID 자격 증명을 가진 유형으로 받아들이지만, 이 버킷을 사용한 이후의 모든 애플리케이션 백업 및 복원이 실패합니다.

   c. 새 버킷 이름을 생성하거나 기존 버킷 이름과 선택적 설명을 입력합니다.

> 버킷 이름 및 설명은 백업을 생성할 때 나중에 선택할 수 있는 백업 위치로 나타납니다. 이 이름은 보호 정책 구성 중에도 표시됩니다.

d. S3 서버의 이름 또는 IP 주소를 입력합니다.

e. 이 버킷을 모든 백업의 기본 버킷으로 사용하려면 "이 버킷을 이 프라이빗 클라우드의 기본 버킷으로 설정" 옵션을 선택합니다.

> 이 옵션은 사용자가 만든 첫 번째 버킷에는 나타나지 않습니다.

f. 를 추가하여 계속합니다 자격 증명 정보.

S3 액세스 자격 증명을 추가합니다

언제든지 S3 액세스 자격 증명을 추가할 수 있습니다.

단계

1. Bucket 대화상자에서 * Add * 또는 * Use Existing * 탭을 선택합니다.
   a. Astra Control의 다른 자격 증명과 구별되는 자격 증명의 이름을 입력합니다.
   b. 클립보드의 내용을 붙여 넣어 액세스 ID와 비밀 키를 입력합니다.

다음 단계

Astra Control Center에 로그인하고 클러스터를 추가했으므로 이제 Astra Control Center의 애플리케이션 데이터 관리 기능을 사용할 준비가 되었습니다.

- "사용자 관리"
- "앱 관리를 시작합니다"
- "앱 보호"
- "앱 클론 복제"
- "알림을 관리합니다"
- "Cloud Insights에 연결합니다"
- "사용자 지정 TLS 인증서를 추가합니다"

자세한 내용을 확인하십시오

- "Astra API를 사용합니다"
- "알려진 문제"

클러스터 추가를 위한 사전 요구사항

클러스터를 추가하기 전에 사전 요구 조건이 충족되는지 확인해야 합니다. 또한 자격 검사를 실행하여 클러스터를 Astra Control Center에 추가할 준비가 되었는지 확인해야 합니다.

클러스터를 추가하기 전에 필요한 사항

- ONTAP 9.5 이상이 지원하는 Trident StorageClasses가 있는 OpenShift 4.6 또는 4.7을 실행하는 클러스터
  - 원격 측정 서비스를 실행할 수 있도록 1GB 이상의 RAM을 사용하는 하나 이상의 작업자 노드

> 두 번째 OpenShift 4.6 또는 4.7 클러스터를 관리되는 컴퓨팅 리소스로 추가할 계획이라면 Trident Volume Snapshot 기능이 활성화되어 있는지 확인해야 합니다. 공식 Trident를 참조하십시오 "지침" Trident를 사용하여 볼륨 스냅샷을 활성화하고 테스트합니다.

- Astra Control Center(ACC)를 사용하여 앱을 백업 및 복원하기 위해 백업 ONTAP 시스템에 설정된 고급 사용자 및 사용자 ID입니다. ONTAP 명령줄에서 다음 명령을 실행합니다. 'export policy rule modify -vserver svm0 -policyname default -ruleindex 1 -superuser sys' export-policy rule modify -policyname default-ruleindex 1-anon 65534'(기본값)

자격 검사를 실행합니다

다음 자격 검사를 실행하여 클러스터를 Astra Control Center에 추가할 준비가 되었는지 확인합니다.

단계

1. Trident 버전을 확인합니다.

```
kubectl get tridentversions -n trident
```

Trident가 있으면 다음과 유사한 출력이 표시됩니다.

```
NAME       VERSION
trident    21.04.0
```

Trident가 없으면 다음과 유사한 출력이 표시됩니다.

```
error: the server doesn't have a resource type "tridentversions"
```

> Trident가 설치되지 않았거나 설치된 버전이 최신 버전이 아닌 경우 계속하기 전에 Trident의 최신 버전을 설치해야 합니다. 를 참조하십시오 "Trident 문서" 를 참조하십시오.

2. 스토리지 클래스가 지원되는 Trident 드라이버를 사용하고 있는지 확인합니다. 공급자 이름은 csi.trident.netapp.io` 이어야 합니다. 다음 예를 참조하십시오.

```
kubectl get storageClass -A
NAME                    PROVISIONER                    RECLAIMPOLICY
VOLUMEBINDINGMODE   ALLOWVOLUMEEXPANSION   AGE
ontap-gold (default)    csi.trident.netapp.io          Delete
Immediate           true                   5d23h
thin                    kubernetes.io/vsphere-volume   Delete
Immediate           false                  6d
```

관리자 역할 **kubecononfig**를 생성합니다

단계를 수행하기 전에 시스템에 다음 사항이 있는지 확인하십시오.

- KUBectl V1.19 이상이 설치되었습니다
- 활성 컨텍스트에 대한 클러스터 관리자 권한이 있는 활성 kubecononfig

단계

1. 다음과 같이 서비스 계정을 생성합니다.

   a. Astractrol-service-account.yaML이라는 서비스 계정 파일을 만듭니다.

      필요에 따라 이름 및 네임스페이스를 조정합니다. 여기에서 변경한 경우 다음 단계에서 동일한 변경 사항을 적용해야 합니다.

```
<strong>astracontrol-service-account.yaml</strong>
```

+

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: astracontrol-service-account
  namespace: default
```

   a. 서비스 계정 적용:

```
kubectl apply -f astracontrol-service-account.yaml
```

2. 다음과 같이 클러스터 관리자 권한을 부여합니다.

   a. astracontrol-clusterrobinding.YAML이라는 ClusterRoleBinding 파일을 만듭니다.

      필요에 따라 서비스 계정을 생성할 때 수정된 모든 이름과 네임스페이스를 조정합니다.

```
<strong>astracontrol-clusterrolebinding.yaml</strong>
```

+

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: astracontrol-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: astracontrol-service-account
  namespace: default
```

a. 클러스터 역할 바인딩을 적용합니다.

```
kubectl apply -f astracontrol-clusterrolebinding.yaml
```

3. '<context>'을(를) 설치에 적합한 컨텍스트로 대체하여 서비스 계정 암호를 나열합니다.

```
kubectl get serviceaccount astracontrol-service-account --context
<context> --namespace default -o json
```

출력의 끝은 다음과 유사합니다.

```
"secrets": [
{ "name": "astracontrol-service-account-dockercfg-vhz87"},
{ "name": "astracontrol-service-account-token-r59kr"}
]
```

제탑 배열의 각 요소에 대한 지수는 0으로 시작합니다. 위의 예에서, astractrol-service-account-dockercfg-vhz87 인덱스는 0이고, astracontrol-service-account-token-r59kr의 인덱스는 1이 된다. 출력에서 "token"이라는 단어가 포함된 서비스 계정 이름의 인덱스를 기록해 둡니다.

4. 다음과 같이 kubecononfig를 생성합니다.

a. create-kubecononfig.sh 파일을 만듭니다. 이전 단계에서 기록한 토큰 인덱스가 0이 아니면 다음 스크립트 시작 부분의 토큰 인덱스 값을 올바른 값으로 바꾸십시오.

```
<strong>create-kubeconfig.sh</strong>
```

```
# Update these to match your environment. Replace the value for
TOKEN_INDEX from
# the output in the previous step if it was not 0. If you didn't
change anything
# else above, don't change anything else here.

SERVICE_ACCOUNT_NAME=astracontrol-service-account
NAMESPACE=default
NEW_CONTEXT=astracontrol
KUBECONFIG_FILE='kubeconfig-sa'
TOKEN_INDEX=0

CONTEXT=$(kubectl config current-context)

SECRET_NAME=$(kubectl get serviceaccount ${SERVICE_ACCOUNT_NAME} \
  --context ${CONTEXT} \
  --namespace ${NAMESPACE} \
  -o jsonpath='{.secrets[TOKEN_INDEX].name}')
TOKEN_DATA=$(kubectl get secret ${SECRET_NAME} \
  --context ${CONTEXT} \
  --namespace ${NAMESPACE} \
  -o jsonpath='{.data.token}')

TOKEN=$(echo ${TOKEN_DATA} | base64 -d)

# Create dedicated kubeconfig
# Create a full copy
kubectl config view --raw > ${KUBECONFIG_FILE}.full.tmp

# Switch working context to correct context
kubectl --kubeconfig ${KUBECONFIG_FILE}.full.tmp config use-context
${CONTEXT}

# Minify
kubectl --kubeconfig ${KUBECONFIG_FILE}.full.tmp \
  config view --flatten --minify > ${KUBECONFIG_FILE}.tmp

# Rename context
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  rename-context ${CONTEXT} ${NEW_CONTEXT}

# Create token user
```

```
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  set-credentials ${CONTEXT}-${NAMESPACE}-token-user \
  --token ${TOKEN}

# Set context to use token user
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  set-context ${NEW_CONTEXT} --user ${CONTEXT}-${NAMESPACE}-token-
user

# Set context to correct namespace
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  set-context ${NEW_CONTEXT} --namespace ${NAMESPACE}

# Flatten/minify kubeconfig
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  view --flatten --minify > ${KUBECONFIG_FILE}

# Remove tmp
rm ${KUBECONFIG_FILE}.full.tmp
rm ${KUBECONFIG_FILE}.tmp
```

b. Kubernetes 클러스터에 적용할 명령을 소스 하십시오.

```
source create-kubeconfig.sh
```

5. (* 선택 사항 *) kubeconfig의 이름을 클러스터의 의미 있는 이름으로 바꿉니다. 클러스터 자격 증명을 보호합니다.

```
chmod 700 create-kubeconfig.sh
mv kubeconfig-sa.txt YOUR_CLUSTER_NAME_kubeconfig
```

다음 단계

이제 필수 구성 요소가 충족되었는지 확인했으므로 이제 수행할 준비가 되었습니다 "클러스터를 추가합니다".

자세한 내용을 확인하십시오

- "Trident 문서"
- "Astra API를 사용합니다"

사용자 지정 **TLS** 인증서를 추가합니다

기존의 자체 서명된 TLS 인증서를 제거하고 CA(인증 기관)에서 서명한 TLS 인증서로 바꿀 수 있습니다.

무엇을 '필요로 할거야

- Astra Control Center가 설치된 Kubernetes 클러스터
- 클러스터의 명령 셸에 대한 관리 액세스로 "kubbtl" 명령을 실행합니다
- CA의 개인 키 및 인증서 파일

자체 서명된 인증서를 제거합니다

1. SSH를 사용하여 관리 사용자로 Astra Control Center를 호스팅하는 Kubernetes 클러스터에 로그인합니다.
2. '<ACC-deployment-namespace>'를 Astra Control Center 배포 네임스페이스로 대체하여 현재 인증서와 연결된 TLS 암호를 찾습니다.

```
kubectl get certificate -n <ACC-deployment-namespace>
```

3. 다음 명령을 사용하여 현재 설치된 암호 및 인증서를 삭제합니다.

```
kubectl delete cert cert-manager-certificates -n <ACC-deployment-
namespace>
kubectl delete secret secure-testing-cert -n <ACC-deployment-namespace>
```

새 인증서를 추가합니다

1. 다음 명령을 사용하여 CA의 개인 키 및 인증서 파일로 새 TLS 암호를 만들고 대괄호 <>의 인수를 적절한 정보로 바꿉니다.

```
kubectl create secret tls <secret-name> --key <private-key-filename>
--cert <certificate-filename> -n <ACC-deployment-namespace>
```

2. 다음 명령 및 예제를 사용하여 클러스터 CRD(Custom Resource Definition) 파일을 편집하고 'pec.selfSigned` 값을 'pec.ca.secretName` 으로 변경하여 앞에서 생성한 TLS 암호를 참조합니다.

```
kubectl edit clusterissuers.cert-manager.io/cert-manager-certificates -n
<ACC-deployment-namespace>
....

#spec:
#  selfSigned: {}

spec:
  ca:
    secretName: <secret-name>
```

3. 다음 명령 및 예제 출력을 사용하여 변경 사항이 올바르고 클러스터가 인증서를 검증할 준비가 되었는지 확인하고 "<ACC-deployment-namespace>"를 Astra Control Center 배포 네임스페이스로 대체합니다.

```
kubectl describe clusterissuers.cert-manager.io/cert-manager-
certificates -n <ACC-deployment-namespace>
....

Status:
  Conditions:
    Last Transition Time:  2021-07-01T23:50:27Z
    Message:               Signing CA verified
    Reason:                KeyPairVerified
    Status:                True
    Type:                  Ready
Events:                    <none>
```

4. 다음 예를 사용하여 대괄호 <>의 자리 표시자 값을 적절한 정보로 대체하여 "certificate.yAML" 파일을 작성합니다.

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: <certificate-name>
  namespace: <ACC-deployment-namespace>
spec:
  secretName: <certificate-secret-name>
  duration: 2160h # 90d
  renewBefore: 360h # 15d
  dnsNames:
  - <astra.dnsname.example.com> #Replace with the correct Astra Control
Center DNS address
  issuerRef:
    kind: ClusterIssuer
    name: cert-manager-certificates
```

5. 다음 명령을 사용하여 인증서를 생성합니다.

```
kubectl apply -f certificate.yaml
```

6. 다음 명령 및 예제 출력을 사용하여 인증서가 올바르게 만들어졌는지, 그리고 생성 중에 지정한 인수(예: 이름, 기간, 갱신 기한 및 DNS 이름)를 사용하여 확인합니다.

```
kubectl describe certificate -n <ACC-deployment-namespace>
....

Spec:
  Dns Names:
    astra.example.com
  Duration:  125h0m0s
  Issuer Ref:
    Kind:        ClusterIssuer
    Name:        cert-manager-certificates
  Renew Before:  61h0m0s
  Secret Name:   <certificate-secret-name>
Status:
  Conditions:
    Last Transition Time:  2021-07-02T00:45:41Z
    Message:               Certificate is up to date and has not expired
    Reason:                Ready
    Status:                True
    Type:                  Ready
  Not After:               2021-07-07T05:45:41Z
  Not Before:              2021-07-02T00:45:41Z
  Renewal Time:            2021-07-04T16:45:41Z
  Revision:                1
Events:                    <none>
```

7. 다음 명령 및 예제를 사용하여 새 인증서 암호를 가리키도록 수신 CRD TLS 옵션을 편집합니다. 대괄호 <>의 개체 틀 값을 적절한 정보로 바꿉니다.

```
kubectl edit ingressroutes.traefik.containo.us -n <ACC-deployment-
namespace>
....

# tls:
#    options:
#      name: default
#    secretName: secure-testing-cert
#    store:
#      name: default

 tls:
    options:
      name: default
    secretName: <certificate-secret-name>
    store:
      name: default
```

8. 웹 브라우저를 사용하여 Astra Control Center의 배포 IP 주소로 이동합니다.
9. 인증서 세부 정보가 설치한 인증서의 세부 정보와 일치하는지 확인합니다.
10. 인증서를 내보내고 결과를 웹 브라우저의 인증서 관리자로 가져옵니다.

Astra Control Center에 대한 질문과 대답

이 FAQ는 질문에 대한 간단한 답변을 찾는 경우에 도움이 될 수 있습니다.

개요

다음 섹션에서는 Astra Control Center를 사용할 때 나타날 수 있는 몇 가지 추가 질문에 대한 답변을 제공합니다. 자세한 내용은 astra.feedback@netapp.com 으로 문의하십시오

Astra Control Center에 액세스할 수 있습니다

- Astra Control URL은 무엇입니까? *

Astra Control Center는 로컬 인증과 각 환경에 고유한 URL을 사용합니다.

URL의 경우 브라우저에서 Astra_control_center_min YAML 사용자 정의 리소스 정의(CRD) 파일(Astra Control Center 설치 시)의 spec.astraAddress 필드에 설정한 FQDN(정규화된 도메인 이름)을 입력합니다. 이메일은 Astra_control_center_min YAML CRD의 spec.email 필드에 설정한 값입니다.

- 평가판 라이센스를 사용하고 있습니다. 전체 라이센스로 변경하는 방법은 무엇입니까? *

NetApp 라이센스 파일(NLF)을 받아 전체 라이센스로 쉽게 변경할 수 있습니다.

- 단계 *

- 왼쪽 탐색 창에서 * 계정 * > * 라이센스 * 를 선택합니다.
- 라이선스 추가 * 를 선택합니다.
- 다운로드한 라이센스 파일을 찾아 * 추가 * 를 선택합니다.
- 평가판 라이센스를 사용하고 있습니다. 앱을 관리할 수 있습니까? *

예. 평가판 라이센스를 사용하여 관리 앱 기능을 테스트할 수 있습니다.

Kubernetes 클러스터를 등록하는 중입니다

- Astra Control에 추가한 후 Kubernetes 클러스터에 작업자 노드를 추가해야 합니다. 어떻게 해야 합니까? *

새 작업자 노드를 기존 풀에 추가할 수 있습니다. 이러한 정보는 Astra Control에서 자동으로 발견됩니다. Astra Control에서 새 노드가 보이지 않으면 새 작업자 노드가 지원되는 이미지 유형을 실행하고 있는지 확인합니다. kubeck get nodes 명령을 사용하여 새 작업자 노드의 상태를 확인할 수도 있습니다.

- 클러스터를 올바르게 관리하려면 어떻게 해야 합니까? *
  1. "Astra Control에서 애플리케이션을 관리합니다".
  2. "Astra Control에서 클러스터 관리를 해제합니다".
- Astra Control에서 Kubernetes 클러스터를 제거한 후 애플리케이션과 데이터는 어떻게 됩니까? *

Astra Control에서 클러스터를 제거해도 클러스터의 구성(애플리케이션 및 영구 스토리지)은 변경되지 않습니다. Astra Control 스냅샷 또는 해당 클러스터의 애플리케이션 백업을 복구할 수 없습니다. Astra Control에서 생성한 영구 스토리지 백업은 Astra Control 내에 남아 있지만 복구할 수 없습니다.

> 다른 방법을 통해 클러스터를 삭제하기 전에 항상 Astra Control에서 클러스터를 제거하십시오. Astra Control에서 관리하는 다른 도구를 사용하여 클러스터를 삭제하면 Astra Control 계정에 문제가 발생할 수 있습니다.

- Astra Control에서 Kubernetes 클러스터를 제거할 때 NetApp Trident가 제거됩니까? *

Astra Control에서 제거할 때 클러스터에서 Trident가 제거되지 않습니다.

응용 프로그램 관리

- Astra Control이 응용 프로그램을 배포할 수 있습니까? *

Astra Control은 애플리케이션을 배포하지 않습니다. 응용 프로그램은 Astra Control 외부에서 배포해야 합니다.

- Astra Control에서 관리를 중지한 후 응용 프로그램은 어떻게 됩니까? *

기존 백업 또는 스냅샷이 삭제됩니다. 애플리케이션과 데이터는 사용 가능한 상태로 유지됩니다. 관리되지 않는 응용 프로그램 또는 해당 응용 프로그램에 속한 백업 또는 스냅숏에는 데이터 관리 작업을 사용할 수 없습니다.

- Astra Control이 NetApp이 아닌 스토리지에 있는 애플리케이션을 관리할 수 있습니까? *

아니요 Astra Control은 NetApp이 아닌 스토리지를 사용하는 애플리케이션을 검색할 수 있지만, NetApp이 아닌 스토리지를 사용하는 애플리케이션은 관리할 수 없습니다.

"Astra Control 자체를 관리해야 하나요?" "아닙니다. Astra Control 자체는 "시스템 앱"이기 때문에 관리하지 말아야 합니다.

데이터 관리 작업

- 내 계정에 생성하지 않은 스냅샷이 있습니다. 어디에서 왔습니까? *

일부 상황에서는 Astra Control이 백업, 클론 또는 복원 프로세스의 일부로 스냅샷을 자동으로 생성합니다.

- My Application은 여러 PVS를 사용합니다. Astra Control이 이러한 모든 PVC의 스냅샷 및 백업을 수행할까요? *

예. Astra Control의 애플리케이션에 대한 스냅샷 작업에는 애플리케이션의 PVC에 바인딩된 모든 PVS의 스냅샷이 포함됩니다.

- Astra Control에서 생성한 스냅샷을 다른 인터페이스 또는 객체 스토리지를 통해 직접 관리할 수 있습니까? *

아니요 Astra Control에서 생성한 스냅샷 및 백업은 Astra Control에서만 관리할 수 있습니다.

저작권 정보

Copyright © 2022 NetApp, Inc. All rights reserved. 미국에서 인쇄됨 본 문서의 어떤 부분도 저작권 소유자의 사전 서면 승인 없이는 어떠한 형식이나 그래픽, 전자적 또는 기계적 수단(사진 복사, 레코딩 등)으로도 저작권 소유자의 사전 서면 승인 없이 전자 검색 시스템에 저장 또는 저장.

NetApp이 저작권을 가진 자료에 있는 소프트웨어에는 아래의 라이센스와 고지 사항이 적용됩니다.

본 소프트웨어는 NetApp에 의해 "있는 그대로" 제공되며 상품성 및 특정 목적에 대한 적합성에 대한 명시적 또는 묵시적 보증을 포함하여 이에 제한되지 않고, 어떠한 보증도 하지 않습니다. NetApp은 대체품 또는 대체 서비스의 조달, 사용 불능, 데이터 손실, 이익 손실, 또는 파생적 손해(소계 물품 또는 서비스의 조달, 사용 손실, 데이터 또는 수익 손실, 계약, 엄격한 책임 또는 불법 행위(과실 또는 그렇지 않은 경우)에 관계없이 어떠한 책임도 지지 않으며, 이는 이러한 손해의 가능성을 사전에 알고 있던 경우에도 마찬가지입니다.

NetApp은 본 문서에 설명된 제품을 언제든지 예고 없이 변경할 권리를 보유합니다. NetApp은 NetApp의 명시적인 서면 동의를 받은 경우를 제외하고 본 문서에 설명된 제품을 사용하여 발생하는 어떠한 문제에도 책임을 지지 않습니다. 본 제품의 사용 또는 구입의 경우 NetApp에서는 어떠한 특허권, 상표권 또는 기타 지적 재산권이 적용되는 라이센스도 제공하지 않습니다.

본 설명서에 설명된 제품은 하나 이상의 미국 특허 또는 해외 특허, 해외 특허, 해외 특허, 해외 특허, 해외 특허, 해외 특허, 해외 특허, 미국 출원 중인 특허로 보호됩니다.

권리 제한 표시: 정부에 의한 사용, 복제 또는 공개에는 DFARS 252.277-7103(1988년 10월) 및 FAR 52-227-19(1987년 6월)의 기술 데이터 및 컴퓨터 소프트웨어의 권리(Rights in Technical Data and Computer Software) 조항의 하위 조항 (c)(1)(ii)에 설명된 제한사항이 적용됩니다.

상표 정보

NETAPP, NETAPP 로고 및 에 나열된 마크는 NetApp에 있습니다 http://www.netapp.com/TM 는 NetApp, Inc.의 상표입니다. 기타 회사 및 제품 이름은 해당 소유자의 상표일 수 있습니다.